TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics



LABORATORY WORK NO. 1

Concurrency in Practice: Multithreading, Shared State and I/O Performance

Course: Network Programming

Author:
Frunza Damian
Student of group: FAF-243

Verified by:
Cristofor Fistic

Chișinău — 2026

Contents

1 Introduction and Objectives....3

2 Hardware and Software Environment....3

2.1 Architectural Implications....3

2.2 Measurement Methodology....4

3 Theoretical Background....4

3.1 Concurrency and Parallelism....4

3.2 Speed-up, Efficiency and Amdahl's Law....4

3.3 Shared State, Race Conditions and Synchronisation....5

3.4 Producer–Consumer Pattern and Bounded Queues....5

3.5 CPU-Bound and I/O-Bound Work....5

4 Implementation....5

4.1 Dataset....6

4.2 Task 1a: Splitting One File Among Threads....6

4.3 Task 1b: Three Ways to Count....6

4.4 Task 1c: One Reader, N Workers, Bounded Queue....6

4.5 Task 2: Many Files on Slow Storage....7

5 Experimental Results....7

5.1 Task 1a: Counting with Threads....7

5.2 Task 1b: Three Ways to Count....8

5.3 Task 1c: Producer and Consumers....10

5.4 Task 2: Many Files on Slow Storage....11

5.4.1 USB Drive with Replug....11

5.4.2 Reference: Internal NVMe SSD....11

6 Task Questions....12

6.1 Task 1 Questions....12

6.1.1 Question 1: At which number of threads does the speed-up stop growing?....12

6.1.2 Question 2: Why do 32 and 64 threads not help?....13

6.1.3 Question 3: Does the peak memory depend on the number of threads? Why?....13

6.1.4 Question 4: Rank the three versions of Task 1b by time....13

6.1.5 Question 5: Compare Task 1c with Task 1a....13

6.1.6 Question 6: What would happen with batches of a single line?....14

6.2 Task 2 Questions....14

6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread....14

6.2.2 Question 2: On the USB drive, do more threads help?....14

7 Control Questions....15

7.1 What is the difference between a physical core and a logical processor?....15

7.2 How did you make sure that no line was counted twice or skipped at the….....15

7.3 Explain step by step how unsynchronised shared counters lose updates....15

7.4 Why are per-thread counters faster than shared counters protected by a….....15

7.5 What does "I/O-bound" mean for a USB drive, and for a server waiting….....16

7.6 What is the difference between dividing work into equal parts in….....16

8 Conclusions....16

9 Source Code and Build Instructions....17

9.1 Build....17

9.2 Generating the Data....17

9.3 Reproducing the Measurements....17

1 Introduction and Objectives

Modern processors contain many cores, but a program benefits from them only if its work is divided among several threads. Whether this really makes it faster depends on the hardware and on how the threads share data and devices. This laboratory work measures that on a real machine with one simple job: counting how many of 50 000 000 integers stored in a text file are negative, positive or zero.

The objectives of the laboratory work are:

1. **Thread scalability (Task 1a):** split one file among 1, 2, 4, 8, 16, 32 and 64 threads, measure time, speed-up and peak memory, and relate the point where the speed-up stops growing to the number of physical cores and logical processors.
2. **Shared state and synchronisation (Task 1b):** count with unsynchronised shared counters, with shared counters protected by a lock and with per-thread counters; find out which versions are correct and what each of them costs.
3. **Producer–consumer pipeline (Task 1c):** connect one reader thread and N worker threads through a bounded queue, compare the result with Task 1a, and study the effect of the batch size and the queue capacity.
4. **CPU-bound and I/O-bound work (Task 2):** process 8 files stored on a USB flash drive with the operating-system file cache bypassed, compare with the internal SSD, and explain when more threads help an I/O-bound program.

The program is written in **C# / .NET 10**. The source, the build instructions and the commands used for every measurement are given in Section 9. The raw measurements are in the CSV files of the `results` folder, and all charts in this report were drawn from them.

2 Hardware and Software Environment

All measurements were made directly on the laptop described in Table 1 (no virtual machine).

Table 1: Hardware and software environment

Component	Specification
CPU	13th Gen Intel Core i7-13700H (hybrid: 6 Performance-cores with Hyper-Threading, up to 5.0 GHz, and 8 Efficient-cores, up to 3.7 GHz; 24 MB L3 cache)
Cores / logical processors	**14 cores** (6 P + 8 E) / **20 logical processors** (6 × 2 + 8 × 1), as reported by Task Manager
RAM	32 GB DDR4-3200 (SODIMM, 2 of 2 slots used; 31.7 GB usable)
Internal disk	WD PC SN560 SDDPNQE-1T00-1102, 1 TB NVMe SSD
USB flash drive	Generic “USB Flash Disk”, 3.75 GB, FAT32. The USB version was not identified; the measured sequential read speed is about **20 MB/s** (Section 5.4), far below the 5 Gbit/s of USB 3.0, so the drive behaves like an old, slow USB 2.0 device
Operating system	Windows 11, version 10.0.26200, x64
Language / runtime	C# / .NET 10, runtime 10.0.14; workstation non-concurrent GC; tiered compilation off (every method is fully optimised from its first call)

2.1 Architectural Implications

The i7-13700H is a *hybrid* processor, so its 20 logical processors are not 20 equal cores. This shapes what the scaling curves can look like:

- **Up to 6 threads** each thread can have a fast P-core to itself, so an almost linear speed-up is expected.
- **From 7 to 14 threads** the new threads land on the 8 slower E-cores, each of which adds less than a P-core.

- **From 15 to 20 threads** the new threads are the second hardware threads (Hyper-Threading) of the P-cores: they share the execution units and caches of a core that is already busy and add only a fraction of a core each.
- **Beyond 20 threads** there is no free hardware left; extra threads only take turns on the same 20 logical processors.
- The data file (369.4 MB) is small compared with the 32 GB of RAM, so after a warm-up run Task 1 reads it from the OS page cache and not from the SSD.

2.2 Measurement Methodology

- **Data:** `numbers.txt`, 50 000 000 pseudo-random integers in [−1 000 000, 1 000 000], 369.4 MB, generated with the fixed seed 2026. True totals: **negative 25 000 383, positive 24 999 592, zero 25, total 50 000 000**. Every task compares its result with them and prints OK or WRONG (column `correct` of the CSV files).
- **Threads** are OS threads created with `new Thread(...)`; the program divides the work itself (no thread pool, no `Parallel.For`).
- **Time** is wall-clock time (`Stopwatch`) from creating the threads until all of them have finished and the totals are added up.
- **Peak memory** is the peak working set of the process (`Process.PeakWorkingSet64`). Every configuration runs in a fresh child process, so the peak belongs to that configuration only. About 27 MiB of it is the .NET runtime itself.
- **Repetitions:** Task 1a 5 runs per thread count; Task 1b sweep 3 runs per version and thread count; Task 1c 3 runs; the extra Task 1c experiments 1–3 runs (Table 7). Runs of the different thread counts were interleaved, and tables and charts show the **median**. Task 1a and 1c start with one unrecorded warm-up run that loads the file into the page cache. Task 2 was run once per thread count (see Section 4.5).
- **Limits of the measurements:** the series are short and were taken on a laptop under Windows, so differences of about 10 % between runs are normal (for example, Task 1a with 64 threads took between 76 and 103 ms, and Task 1c with one worker between 0.84 and 1.07 s). Conclusions in this report rely on differences much larger than that.

3 Theoretical Background

3.1 Concurrency and Parallelism

Concurrency means that several tasks are in progress during the same period; on a single processor the scheduler interleaves them. *Parallelism* means that tasks run at the same instant on different hardware. Threads always give concurrency, but they run in parallel only up to the number of logical processors (20 here), and at full speed only while every thread has a physical core to itself.

3.2 Speed-up, Efficiency and Amdahl's Law

If T_1 is the time with one thread and T_p the time with p threads, the speed-up and the parallel efficiency are

$$S(p) = T_1 / T_p, \qquad E(p) = S(p) / p$$

If only a fraction f of the work can run in parallel and the rest, 1 − f, is sequential (starting threads, adding up totals, reading input on one thread), Amdahl's law bounds the speed-up:

$$S(p) = 1 / ((1 - f) + f / p) \le 1 / (1 - f)$$

Even with unlimited threads the time cannot fall below the time of the sequential part. In practice the speed-up is lower still, because extra threads have costs of their own: creating and scheduling them and competing for shared resources.

3.3 Shared State, Race Conditions and Synchronisation

A *data race* occurs when two threads access the same memory location at the same time, at least one of them writes, and nothing orders the accesses. The statement `counter++` is not one indivisible operation but three steps: load the value into a register, add 1, store the register back. If two threads interleave these steps, one store overwrites the other and an increment is lost (a *lost update*). The usual remedies are:

- **Mutual exclusion** (`lock`): only one thread at a time runs the critical section. It is correct, but the threads are serialised, every acquire and release costs atomic operations, and a waiting thread may be put to sleep and woken up by the OS.
- **Atomic instructions** (`Interlocked.Increment`, which compiles to `lock xadd` on x86): each increment is indivisible without a lock, but all threads still write to the same memory.
- **No sharing:** each thread counts into private variables and the results are added once at the end (a *reduction*). There is nothing to synchronise while counting.

Shared data also has a hardware cost. Caches work with 64-byte *cache lines*; a line written by several cores must move from core to core on every write (cache-coherence traffic), even when the accesses are correctly synchronised.

3.4 Producer–Consumer Pattern and Bounded Queues

In the producer–consumer pattern, producers put work items into a queue and consumers take them out. A *bounded* queue of capacity C makes the producer wait when C items are already waiting (**back-pressure**, which limits memory use) and makes the consumers wait when it is empty. Every `Add` and `Take` is a synchronised operation, so items are grouped into **batches** to spread that cost over many lines. Unlike a split made in advance, a queue balances the load by itself: a free consumer simply takes the next item.

3.5 CPU-Bound and I/O-Bound Work

A program is *CPU-bound* when its time goes on computation, so more cores make it faster. It is *I/O-bound* when most of its time is spent waiting for a device or the network. Concurrency helps I/O-bound work only if the waits are **independent** and can overlap (for example many network requests, each waiting for a different server). It does not help when all threads queue for **one device that is already at its bandwidth limit**.

4 Implementation

The measurements use the Lab1 benchmark program from the author's repository PR_26 (folder `Lab1`), which was built and run on the machine of Section 2. It is one console program in which the task and its parameters are chosen with command-line arguments (Section 9). Threads are always real OS threads created with `new Thread(...)`. Table 2 lists the source files.

Table 2: Structure of the source code

File	Contents
`Program.cs`	command dispatch and `help` text
`Args.cs`	command-line parser (`--key value`, `--flag`)
`Numbers.cs`	line parser, `Counts`, splitting a file into line-aligned byte ranges
`Measure.cs`	peak memory, the expected totals, the machine-readable result line

File	Contents
`DataTools.cs`	`gen` (data file), `split` (8 parts), `info` (hardware)
`Task1a.cs`	Task 1a – each thread reads and parses its own part of the file
`Task1b.cs`	Task 1b – unsynchronised / lock / per-thread counters (+ `Interlocked` as an extra)
`Task1c.cs`	Task 1c – one reader, N workers, bounded `BlockingCollection`
`Task2.cs`	Task 2 – one file = one unit of work, `File.ReadAllBytes`, files divided among threads
`Bench.cs`	runs a task for several thread counts, each run in a fresh process, writes CSV
`Charts.cs`	turns CSV files into SVG charts (not used here: the charts of this report were drawn with Python)

4.1 Dataset

`lab1 gen` writes `data/numbers.txt` (50 000 000 integers, one per line) and `numbers.txt.counts` with the true totals. For Task 2, `lab1 split` cuts the file, in order, into `part_1.txt … part_8.txt` (6 250 000 lines, 46.2 MB each) plus `expected.counts`. Every run checks that $N_{negative} + N_{positive} + N_{zero} = 50\,000\,000$ and compares each total with the expected value.

4.2 Task 1a: Splitting One File Among Threads

1. Thread i of N receives the raw byte range [size·i/N, size·(i+1)/N).
2. Both ends of the range are moved forward to the next line start (`Numbers.AlignToLineStart`), so a line belongs to the thread whose range contains its first byte.
3. The thread reads its range in 256 KiB chunks with positional reads (`RandomAccess.Read`), parses the bytes directly and counts into local variables.
4. The main thread waits for all threads (`Join`) and adds up the per-thread totals.

4.3 Task 1b: Three Ways to Count

The numbers are first loaded into an `int[]` (not measured). Then the threads each count their share of the array in one of these versions:

1. **Shared, no synchronisation:** all threads increment the same three shared counters. The main experiment runs this version 3 times to show that its result changes from run to run.
2. **Shared + lock:** every increment is done inside `lock (gate) { … }`.
3. **Per-thread counters:** each thread counts into its own local variables; the main thread adds them after `Join`.
4. **Extra –** shared + `Interlocked`**:** atomic increments without a lock, added for comparison.

The main experiment uses 16 threads; the sweep repeats all four versions with 1, 2, 4, 8, 16, 32 and 64 threads.

4.4 Task 1c: One Reader, N Workers, Bounded Queue

One reader thread reads the file with `StreamReader.ReadLine()`, collects the lines into `string[10 000]` batches and adds them to a `BlockingCollection<string[]>` with capacity 100 (a bounded blocking FIFO queue). At the end of the file it closes the queue with `CompleteAdding()`. N worker threads take batches until the queue is closed and empty, parse the lines and count into local variables. The program also records how long the reader waited on a full queue, how long the workers waited on an empty queue and the longest the queue ever got. Batch size and capacity are parameters (`batch`, `--capacity`, where 0 means unbounded), used for question 6.

4.5 Task 2: Many Files on Slow Storage

A file is the unit of work: a thread reads it with **one `File.ReadAllBytes` call into one `byte[]`**, then parses the buffer and counts. The 8 files are divided into equal contiguous groups (1 thread: 8 files; 2 threads: 4 + 4; 4 threads: 2 + 2 + 2 + 2). For every file the program measures how long the read and the parse took.

Two series were measured:

- **USB flash drive with replug:** the 8 files were copied to the drive and read only from there. Before every run the drive was ejected, unplugged and plugged in again (`bench --pause` waits for Enter), so nothing could come from the file cache. One run per thread count (`results/task2.csv`).
- **Reference, internal NVMe SSD:** the same program on the SSD with the files already in the page cache (“warm”), one run per thread count (`results/task2-ssd-warm.csv`).

A cold-cache SSD series and a USB series without replug were not measured: both rely on `posix_fadvise` to drop files from the page cache, which exists only on Linux.

5 Experimental Results

5.1 Task 1a: Counting with Threads

Table 3 shows the median of 5 runs for every number of threads; efficiency is E = S/p. The totals were correct in all 35 runs. Figures 1–3 show Task 1a together with Task 1c so that the two can be compared (question 5).

Table 3: Task 1a – time, speed-up, efficiency and peak memory vs. number of threads

Threads	Time, s	Speed-up	Efficiency	Peak memory, MiB	Totals correct
1	0.491	1.00×	100.0 %	26.8	yes
2	0.256	1.91×	95.7 %	27.1	yes
4	0.144	3.41×	85.2 %	27.6	yes
8	0.107	4.58×	57.2 %	28.6	yes
16	0.080	6.13×	38.3 %	31.3	yes
32	0.080	6.11×	19.1 %	35.5	yes
64	0.079	6.23×	9.7 %	44.1	yes

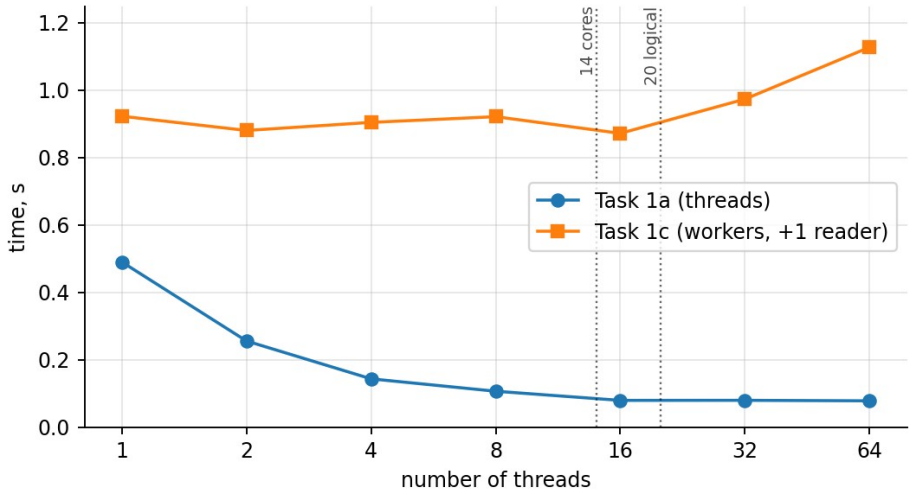


Figure 1: Task 1a and Task 1c – wall-clock time vs. number of threads

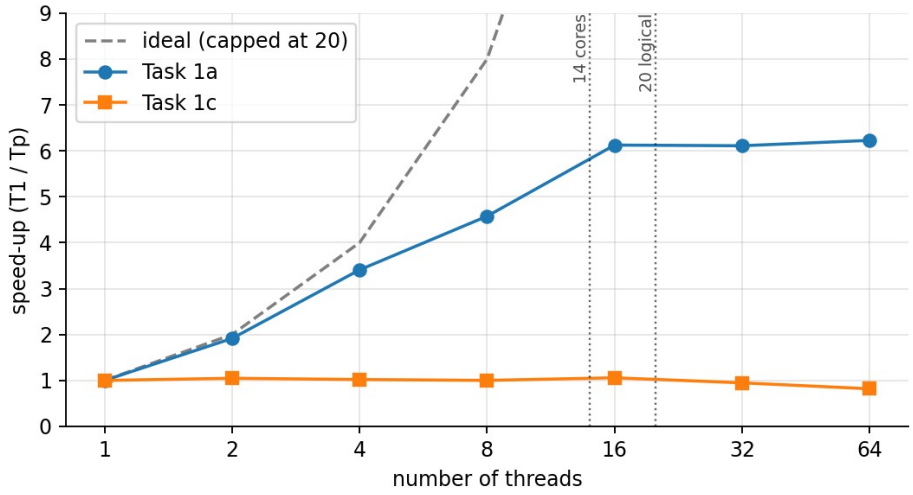


Figure 2: Task 1a and Task 1c – speed-up vs. number of threads (dotted lines: 14 cores and 20 logical processors)

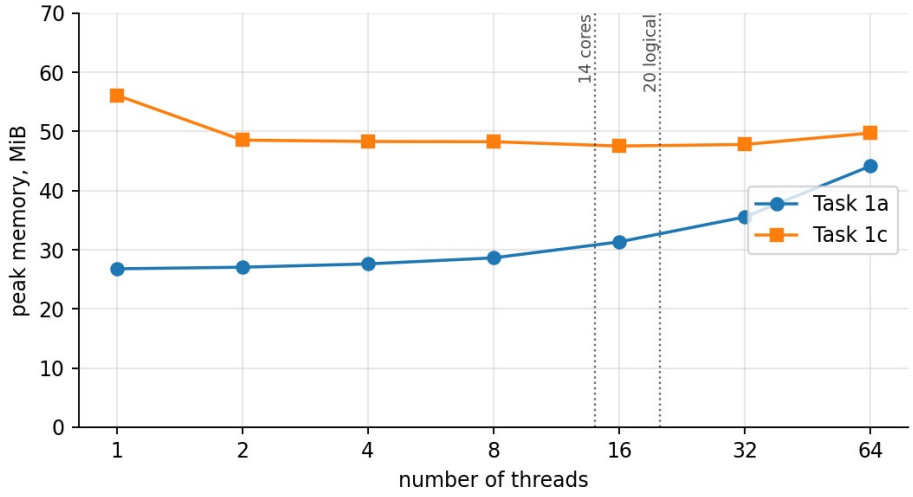


Figure 3: Task 1a and Task 1c – peak memory vs. number of threads

5.2 Task 1b: Three Ways to Count

Table 4 shows the main run with 16 threads (`results/task1b.csv`). The unsynchronised version was run three times.

Table 4: Task 1b – main run with 16 threads

Version	Time, s	Negative	Positive	Zero	Total	Correct
1. shared, no synchronisation – run 1	0.231	7 144 607	7 096 284	25	14 240 916	**no**, 35 759 084 updates lost
1. shared, no synchronisation – run 2	0.116	5 834 924	5 826 871	25	11 661 820	**no**, 38 338 180 updates lost
1. shared, no synchronisation – run 3	0.116	5 491 932	5 472 137	25	10 964 094	**no**, 39 035 906 updates lost
2. shared + lock	2.990	25 000 383	24 999 592	25	50 000 000	yes
extra: shared + `Interlocked`	1.741	25 000 383	24 999 592	25	50 000 000	yes
3. per-thread counters	**0.031**	25 000 383	24 999 592	25	50 000 000	yes

The same four versions were also run with 1–64 threads (`results/task1b-sweep.csv`, median of 3 runs). The results are in Table 5 and Figures 4–6 (time on a logarithmic scale).

Table 5: Task 1b – time (s) of the four versions vs. number of threads

Threads	unsync, s	lock, s	`Interlocked`, s	per-thread, s
1	0.192 (correct)	0.721	0.316	0.180
2	0.140 (wrong)	1.831	0.545	0.099
4	0.185 (wrong)	6.937	0.660	0.060
8	0.222 (wrong)	8.135	1.786	0.055
16	0.313 (wrong)	9.972	1.788	0.033
32	0.561 (wrong)	8.072	1.133	0.050
64	0.687 (wrong)	8.091	1.720	0.034

The unsynchronised version is correct only with one thread; with 2 or more threads it was wrong in every run, losing between 7.9 and 40.6 million of the 50 million updates. Peak memory is about 410 MiB in every case (409.7–410.8 MiB): it is the 200 MB `int[]` plus the memory used while loading it, and does not depend on the version or the number of threads. With only 3 runs per point the per-thread curve is noisy (for example 50 ms at 32 threads against 33 ms at 16 and 64).

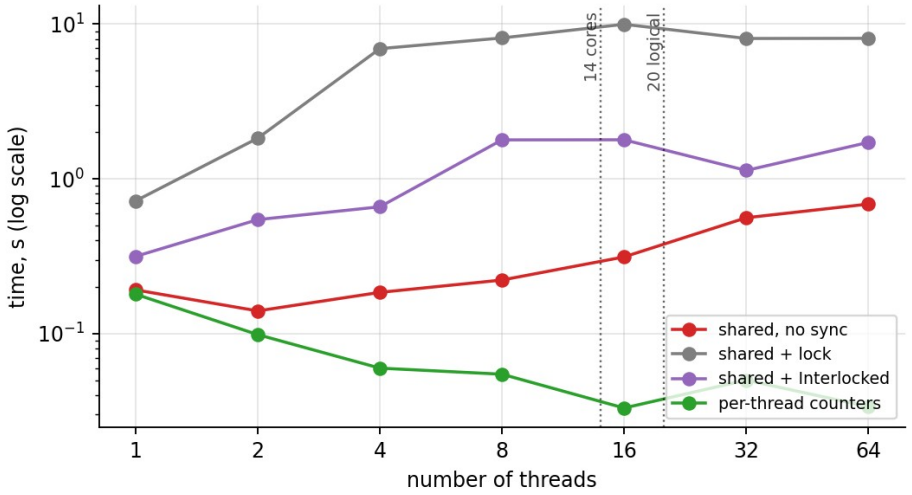


Figure 4: Task 1b – time of the four versions vs. number of threads (logarithmic scale)

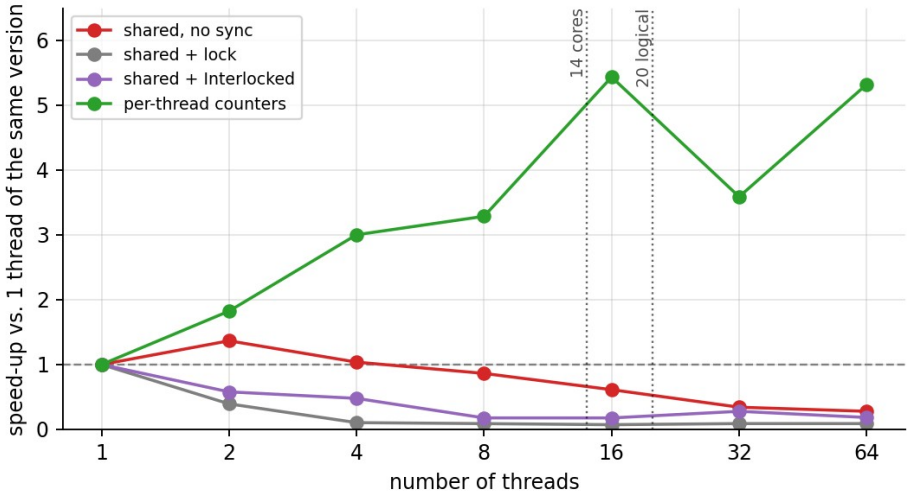


Figure 5: Task 1b – speed-up of the four versions vs. number of threads

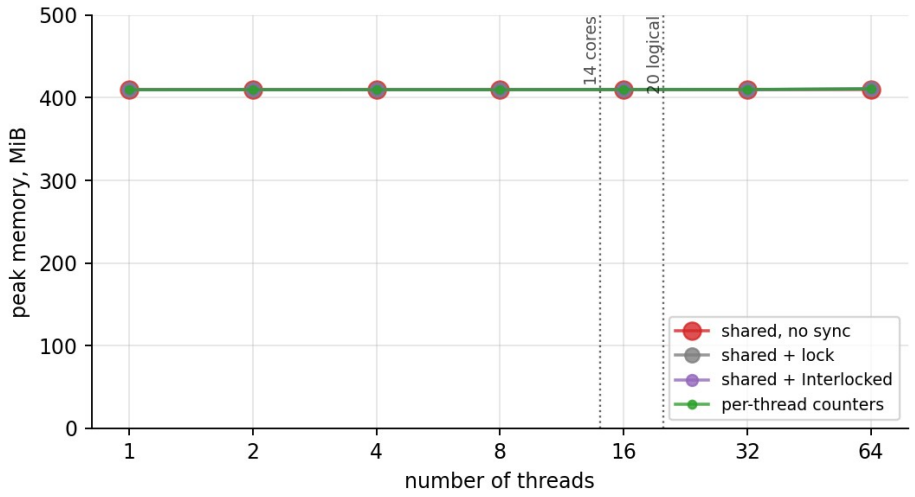


Figure 6: Task 1b – peak memory vs. number of threads

5.3 Task 1c: Producer and Consumers

Table 6 shows the median of 3 runs with batches of 10 000 lines and queue capacity 100. The orange series in Figures 1–3 are the same data.

Table 6: Task 1c – time, peak memory and queue statistics vs. number of workers

Workers	Time, s	Speed-up	Peak mem., MiB	Reader waited on full queue	Longest queue, batches	Avg. worker wait on empty queue, s	Correct
1	0.923	1.00×	56.1	0	3–15	0.32–0.38	yes
2	0.881	1.05×	48.5	0	3–4	0.54–0.69	yes
4	0.905	1.02×	48.3	0	3–4	0.75–0.91	yes
8	0.922	1.00×	48.3	0	2–7	0.78–0.97	yes
16	0.872	1.06×	47.5	0	3–4	0.80–1.01	yes
32	0.974	0.95×	47.8	0	4	0.87–1.09	yes
64	1.128	0.82×	49.7	0	5–6	1.10–1.11	yes

Table 7 shows the extra experiments for question 6 (`task1c-batch1.csv`, `task1c-unbounded.csv`, `task1c-batch1-unbounded.csv`).

Table 7: Task 1c – effect of the batch size and of the queue capacity

Configuration	Workers	Time, s	Peak memory, MiB	Longest queue
batch 1 line, capacity 100	1	20.97	39.9	100 (queue full 10 858 times)
batch 1 line, capacity 100	4	55.65	40.1	100 (queue full 1 277 times)
batch 1 line, capacity 100	16	72.80	40.5	100 (queue full 12 times)
batch 10 000, **unbounded**	1 / 4 / 16	0.799 / 0.811 / 0.830	48.7 / 48.3 / 49.1	4–5
batch 1 line, **unbounded**	1	11.52	42.3	**21 782**

Batch 1 with capacity 100 and with an unbounded queue: 1 run each. Batch 10 000 unbounded: median of 3 runs.

5.4 Task 2: Many Files on Slow Storage

5.4.1 USB Drive with Replug

Table 8: Task 2 on the USB drive, drive replugged before every run

Threads	Time, s	Speed-up	Peak mem., MiB	Reading (thread-time sum), s	Parsing (sum), s	Overall MB/s	Correct
1	**18.99**	1.00×	245.7	18.54	0.44	19.5	yes
2	**18.55**	1.02×	248.2	34.32	0.48	19.9	yes
4	**18.37**	1.03×	380.4	59.24	0.46	20.1	yes

The 8 files total 369.5 MB. Reading them took 18.5 s with one thread, which is about 20 MB/s, while parsing all of them took only 0.44–0.48 s. With 2 and 4 threads the sum of the time spent inside `ReadAllBytes` grew to 34.3 s and 59.2 s, but the wall-clock time stayed at about 18.5 s. The three runs were all correct.

5.4.2 Reference: Internal NVMe SSD

Table 9: Task 2 on the internal NVMe SSD, files in the page cache (warm)

Threads	Time, s	Speed-up	Reading (sum), ms	Parsing (sum), ms	Peak mem., MiB	Overall MB/s	Correct
1	0.602	1.00×	121	475	245.8	613.8	yes
2	0.307	1.96×	143	460	380.1	1203.7	yes
4	0.185	3.26×	206	508	380.2	2001.2	yes

On the SSD, reading all 369.5 MB takes only 0.12 s of the 0.60 s single-thread time (the files come from the page cache), and parsing takes 0.47 s. The work is therefore **CPU-bound** there, and 2 and 4 threads speed it up by 1.96× and 3.26×, similar to Task 1a.

Peak memory grows with the number of threads on both devices (about 246 MiB with 1 thread, up to 380 MiB with 4), because every thread holds a whole 46 MB file in memory while it parses it. A 46 MB array is a large object that .NET frees only in a full collection, so several finished buffers can still be resident when the next ones are allocated; the exact peak therefore varies from run to run.

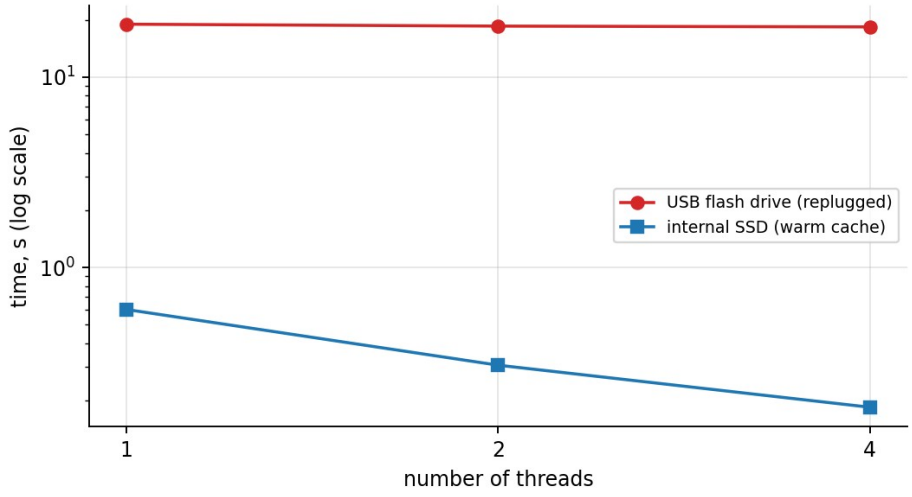


Figure 7: Task 2 – time on the USB drive and on the SSD (logarithmic scale)

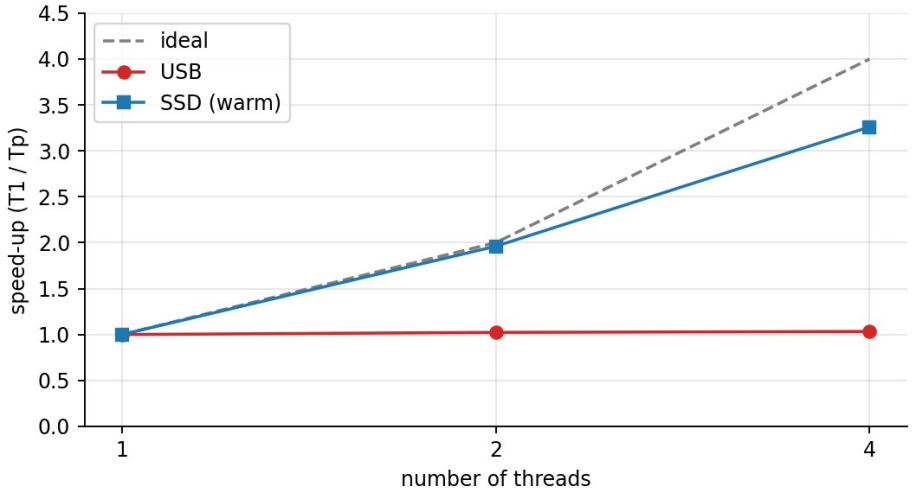


Figure 8: Task 2 – speed-up on the USB drive and on the SSD

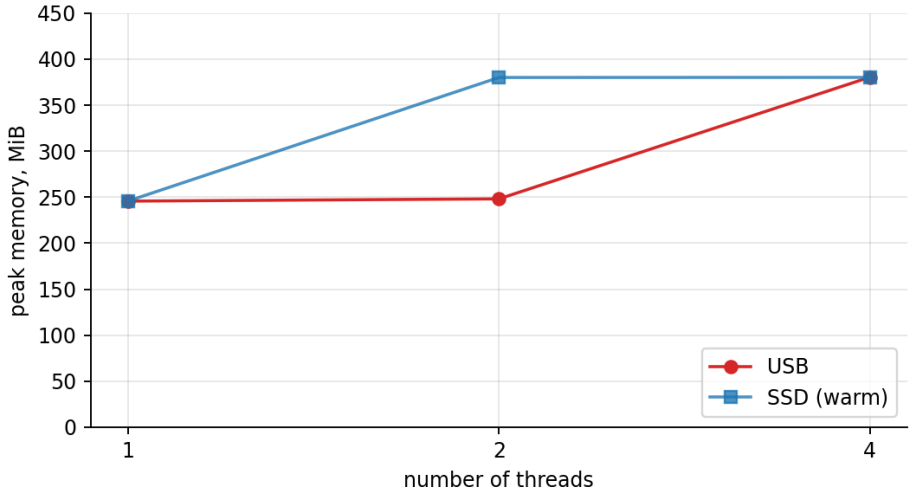


Figure 9: Task 2 – peak memory on the USB drive and on the SSD

6 Task Questions

6.1 Task 1 Questions

6.1.1 Question 1: At which number of threads does the speed-up stop growing? Relate it to physical cores and logical processors.

At about **16 threads**. Up to 4 threads the speed-up is close to ideal (1.91× with 2 threads, 3.41× with 4), because every thread can run alone on a fast P-core. It then grows more slowly: 4.58× with 8 threads and 6.13× with 16. After that it stays flat: 6.11× with 32 and 6.23× with 64 threads, which is within the run-to-run spread. The first 6 threads fill the P-cores; threads 7–14 go to the slower E-cores (up to 3.7 GHz against 5.0 GHz, simpler design); threads 15–20 can only be the second hardware threads of P-cores, which share a core that is already busy and add little. So the 20 logical processors are worth much less than 20 cores, and even the 14 physical cores are not 14 equal cores. Beyond 20 threads there is no hardware left to run them on. The curve is already flat at 16, slightly below 20 logical processors, because the thread counts 17–20 only add Hyper-Threading siblings; since 20 threads were not measured, the exact point between 16 and 32 cannot be located. A possible additional limit is the memory bandwidth needed to stream the 369 MB from the page cache; it was not measured separately. The per-thread version of Task 1b, which only reads an array, shows a similar picture: 3.0× with 4 threads and 5.4× with 16.

6.1.2 Question 2: Why do 32 and 64 threads not help? Are they slower, and why?

They do not help because only 20 threads can run at the same moment; the others wait for a time slice, and the same CPU work is merely cut into more pieces. For Task 1a they are **not slower in the median** (0.080 s with 32 and 0.079 s with 64 threads against 0.080 s with 16), but they are less predictable: the five runs with 64 threads range from 76 to 103 ms, those with 16 threads from 77 to 86 ms. The extra costs exist (creating and joining 48 more threads, more context switches, more 256 KiB buffers and stacks, more boundary searches) but amount to a few milliseconds out of about 80. Where threads compete for a shared resource, extra threads do hurt: Task 1c becomes slower with every extra worker beyond 16 (0.87 s, 0.97 s and 1.13 s with 16, 32 and 64 workers), and the lock and `Interlocked` versions of Task 1b are far slower with many threads than with one.

6.1.3 Question 3: Does the peak memory depend on the number of threads? Why?

Yes, but only slightly and almost linearly: 26.8 MiB with 1 thread, 31.3 MiB with 16 and 44.1 MiB with 64, i.e. about 0.27 MiB for every additional thread. Each thread has its own 256 KiB read buffer, its own stack (only the touched pages count) and some runtime bookkeeping. The file itself is **never loaded as a whole**: each thread streams through its part, so memory does not depend on the file size. The base of about 27 MiB is the .NET runtime and the compiled code.

6.1.4 Question 4: Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

From fastest to slowest with 16 threads (medians of the sweep, Table 5): **per-thread counters 0.033 s < unsynchronised 0.313 s** < `Interlocked` **1.79 s < lock 9.97 s**. (In the single main run of Table 4 the numbers were 0.031, 0.116–0.231, 1.74 and 2.99 s; lock times vary a lot from run to run, but the order never changed.) The **lock**, `Interlocked` and **per-thread** versions are correct. The unsynchronised version is wrong every time, and by a different amount each time: in the main run 35.8, 38.3 and 39.0 million of the 50 million increments were lost (72–78 %), and in the sweep up to 40.6 million (81 %).

- *Unsynchronised:* `counter++` is load, add, store. When two threads load the same value, both store value + 1 and one increment disappears. It is also slower than it looks for a version without locks: the three counters share one 64-byte cache line that all cores keep writing, so the line bounces between cores. The time grows from 0.14 s with 2 threads to 0.69 s with 64. Only `zero` is exact (25), because zeros are so rare that two threads almost never update it at the same moment.
- *Lock:* correct, because every read-modify-write happens inside one critical section, but 50 million lock/unlock pairs are executed and only one thread at a time can be inside. The threads take turns, the lock itself is a contended cache line, and waiting threads may be put to sleep and woken up by the OS. With 16 threads it takes 9.97 s, which is **13.8× slower than one thread** (0.72 s) and about 300× slower than per-thread counting.
- `Interlocked` *(extra):* an atomic `lock xadd` is correct and cheaper than a lock, but every increment still fights for the same cache line: 1.79 s with 16 threads, 5.7× slower than one thread (0.32 s).
- *Per-thread:* each thread counts in its own local variables (CPU registers); nothing is shared while counting and there is one write per thread at the end. There is no synchronisation and no cache-line traffic, so it is correct and scales: 5.4× faster than one thread with 16 threads.

6.1.5 Question 5: Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is much faster: 0.080 s with 16 threads against 0.87 s for the best Task 1c configuration (16 workers), i.e. **about 11× faster**. Even Task 1a with a single thread (0.49 s) beats every Task 1c configuration. Task 1c **does not get faster in any useful way: it is already at its best with 1–2 workers**; the median is 0.92 s with 1 worker, 0.88 s with 2 and 0.87 s with 16, differences smaller than

the spread between runs of the same configuration (0.84–1.07 s for one worker), and it gets slower with 32 and 64 workers (0.97 s and 1.13 s).

The measurements show why. In none of the 21 runs of Table 6 did the reader find the queue full, and the queue never held more than 15 of the 100 allowed batches (usually 3–5). The workers, meanwhile, spent most of the run waiting for an empty queue to be refilled: 0.3 s of 0.9 s with one worker and almost the whole run with 64 workers (1.10 s of 1.13 s). The run time equals the time the reader needs (for example 836 ms of reader time in a run of 836 ms). So the **single reader thread is the bottleneck**. It does the sequential part of the job: reading, finding line ends, converting UTF-8 bytes into UTF-16 `string` objects and allocating one object per line; the console output of a run with 64 workers shows about 2 270 MiB allocated by the process. The parsing done by the workers is the cheap part. By Amdahl's law the time cannot fall below the reader's time of about 0.8–0.9 s, and extra workers only add contention on the queue and wake-ups. In Task 1a, reading and splitting into lines are done in parallel, directly on bytes, using reusable 256 KiB buffers and no per-line objects.

6.1.6 Question 6: What would happen with batches of a single line? And with a queue that has no size limit?

Measured (Table 7). With **batches of one line** every number costs one `Add` and one `Take` on the shared queue (locking, signalling, often waking a sleeping thread): 50 million queue operations instead of 5 000. One worker took **21.0 s, about 23× longer** than with 10 000-line batches (0.92 s), and more workers made it worse: 55.7 s with 4 workers and 72.8 s with 16. The workers fight for the queue's lock and wait nearly all the time (on average 48.7 s of 55.7 s with 4 workers, 70.3 s of 72.8 s with 16). The synchronisation cost per item is larger than the work per item, which is why batching is what makes the queue usable.

With an **unbounded queue** and 10 000-line batches nothing changed (0.80–0.83 s, 48–49 MiB), because the consumers are faster than the producer and the queue never holds more than 5 batches. The limit matters when the producer is faster than the consumers. In the one configuration where the worker fell behind (batch 1 line, 1 worker) the unbounded queue grew to **21 782 entries** instead of stopping at 100 (it also finished sooner, 11.5 s instead of 21.0 s, because the reader never had to wait). With 10 000-line batches and slow workers, the whole file could end up in the queue as strings, gigabytes of memory, up to an out-of-memory error. A bounded queue provides **back-pressure**: it makes the reader wait and keeps memory below 100 × 10 000 lines (a few tens of MB of strings).

6.2 Task 2 Questions

6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?

I/O-bound. With one thread Task 2 took **18.99 s** on the USB drive, against **0.49 s** for Task 1a with one thread (**39× slower**) and 0.60 s for the same Task 2 on the SSD (32× slower), for exactly the same numbers and the same parsing code. Of the thread time, **18.54 s (97.7 %) was spent waiting in** `ReadAllBytes` and only **0.44 s (2.3 %) on parsing**. The parsing time is the same as on the SSD (0.47 s) and similar to Task 1a, so the CPU did not do more work; it waited. The throughput of about 20 MB/s is the limit of this drive, while one core parses roughly 835 MB/s (369.5 MB in 0.44 s), about 43× more. The CPU was idle more than 97 % of the time.

6.2.2 Question 2: On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

No. With 1, 2 and 4 threads the times were 18.99, 18.55 and 18.37 s, a speed-up of only 1.00, 1.02 and 1.03. All threads wait for **the same single device** whose bandwidth is fixed at about 20 MB/s, and one thread reading sequentially already uses all of it. The overall throughput stays at 19.5, 19.9 and 20.1 MB/s, while the sum of the time the threads spend inside `ReadAllBytes` grows from 18.5 s to 34.3 s and 59.2 s: the threads simply wait longer each, because they share the same bandwidth. The tiny gain

of about 0.6 s is about the size of the whole parsing time (0.46 s), which is the only work that can overlap with reading when several threads are present.

Concurrency helps I/O-bound work when the waits are **independent latencies**, for example many network requests, each waiting for a different server while the link and the CPU are idle: the waits overlap. It does not help when all threads queue for **one resource that is already at its throughput limit**, which is exactly the case with a USB flash drive.

7 Control Questions

7.1 What is the difference between a physical core and a logical processor? How did it show in your charts?

A physical core is real execution hardware: its own arithmetic units, registers and L1/L2 caches. A logical processor is what the OS schedules threads on. With Hyper-Threading (SMT) one P-core appears as two logical processors; they have separate register sets but share the core's execution units and caches, so two threads on one core run only a little faster than one thread, not twice as fast. The i7-13700H has 6 P-cores × 2 + 8 E-cores × 1 = 20 logical processors on 14 physical cores, and the E-cores are slower than the P-cores. In the charts (Figure 2) Task 1a scales almost linearly up to 4 threads (3.41×), bends between 8 and 16 threads (4.58× → 6.13×, far from 16×) and is flat from 16 threads on.

7.2 How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?

Each thread gets a raw byte range [size·i/N, size·(i+1)/N). The rule is: *a line belongs to the thread whose range contains its first byte.* Both ends of the range are moved with the same function, `AlignToLineStart(offset)`: if the byte before `offset` is a newline, `offset` is already a line start; otherwise the function moves forward to the byte after the next newline. The aligned end of thread i is computed exactly like the aligned start of thread i + 1, so the aligned ranges fit together with no gap and no overlap, and each thread parses only complete lines. Evidence from the measurements: all 35 runs of Task 1a, with every thread count from 1 to 64, produced exactly 50 000 000 numbers and exactly the true totals (25 000 383, 24 999 592 and 25).

7.3 Explain step by step how unsynchronised shared counters lose updates.

`negative++` is executed as (1) load the value from memory into a register, (2) add 1, (3) store the register back. Even a single `inc [mem]` instruction does these three steps without the `lock` prefix and is not atomic across cores. Example with `negative = 100`:

1. thread A loads 100;
2. thread B loads 100;
3. A computes 101 and stores 101;
4. B computes 101 and stores 101.

Two numbers were counted but the counter grew by one, because B's store overwrote A's update. With 16 threads hitting the same counters millions of times, this happened to 72–78 % of the increments in the main run, and differently in each run.

7.4 Why are per-thread counters faster than shared counters protected by a lock?

With a lock, each of the 50 million increments needs an acquire and a release (atomic instructions on the lock's memory). Only one thread at a time can hold the lock, so the threads run one after another instead of in parallel. The lock and the counters move between the cores' caches on every operation, and a thread that finds the lock taken may be put to sleep and woken up by the OS. Per-thread counters share nothing: they stay in registers, need no atomic instruction and no cache-line transfer, and are

combined once after `Join`. Measured with 16 threads: 0.033 s against 9.97 s (about 300×) in the sweep, and 0.031 s against 2.99 s (about 98×) in the main run.

7.5 What does “I/O-bound” mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?

I/O-bound means that the program spends most of its time waiting for input or output instead of computing.

- For the **USB drive** the limit is the **bandwidth of one device**: a single sequential reader already keeps it busy. More threads only share the same MB/s. Measured: with 1, 2 and 4 threads the total time was 19.0, 18.5 and 18.4 s, although the time the threads spent waiting in total grew from 18.5 to 59.2 s.
- For a **server waiting on the network** the time goes on **latency**: each request waits, for example, 100 ms for a remote machine, while the link and the CPU are mostly idle. These waits are independent of each other, so many requests can wait at the same time and 100 concurrent requests take about as long as one.

Concurrency helps when it overlaps waiting on *independent* resources, and does not help when the threads queue for the *same* saturated resource.

7.6 What is the difference between dividing work into equal parts in advance and taking it from a queue? When is each one better?

- *Static partitioning* (Task 1a, Task 2): each thread knows its part from the start. There is no coordination during the work, no shared state, no allocation per item and good locality, but the total time is the time of the **slowest** part. It is best when the work splits evenly, its size is known in advance (a file you can seek in, an array) and the cores are equal.
- *A shared queue* (Task 1c): threads take the next piece when they are free, which **balances the load automatically**. In the 64-worker run of Task 1c the 5 000 batches were spread between 71 and 84 per worker. A queue is what you need on a hybrid P/E processor, when items have different costs, or when the input is a stream of unknown size that can only be read in order. The price is synchronisation on every take, so items must be batched (batch 1 was about 23× slower), and a single producer can become the bottleneck, as it did in Task 1c.

A queue is better for uneven or unknown work, streaming input or uneven cores; static parts are better for uniform, splittable work.

8 Conclusions

1. **CPU-bound work and the hardware limit.** Task 1a, Task 1b, Task 1c and Task 2 on the internal SSD are CPU-bound. Evidence: (1) the time falls almost in proportion to the number of threads until the cores run out (Task 1a: 3.41× with 4 threads and 6.13× with 16; per-thread counting in Task 1b: 5.4× with 16 threads) and then stays flat, around the 14 physical cores and 20 logical processors of this hybrid CPU; (2) reading the whole data set from the page cache takes only 0.12 s of the 0.60 s single-thread time of Task 2 on the SSD, the rest is parsing.
2. **The sequential part limits the speed-up.** Task 1c is CPU-bound too, but on **one** thread, the reader, so adding workers cannot help (about 0.9 s with 1 to 16 workers, against 0.08 s for Task 1a with 16 threads). This is Amdahl’s law at work.
3. **I/O-bound work.** Task 2 on the USB drive is I/O-bound. Evidence: 97.7 % of the thread time (18.54 s of 18.99 s) is spent waiting in `ReadAllBytes`; the parsing takes the same 0.44–0.48 s as on the SSD; the throughput (about 20 MB/s) is about 43× lower than what one core can parse; and more threads change nothing (1.02–1.03×) because they all share the bandwidth of the same device. The same code on the SSD is CPU-bound and speeds up 3.26× with 4 threads. Whether

a task is CPU-bound or I/O-bound is not a property of the code but of the code on particular hardware.

4. **Synchronisation cost.** Sharing a counter is either wrong (unsynchronised: 72–81 % of the updates lost) or very slow (lock: 13.8× slower than one thread and about 300× slower than per-thread counters with 16 threads). The fast and correct design shares nothing while working and combines the results once at the end.

9 Source Code and Build Instructions

The program is the Lab1 program of the author's repository PR_26; `README.md` in its `Lab1` folder describes every command. It requires the .NET 10 SDK. The commands below are the ones used for the measurements in this report, written for Windows PowerShell (a comma-separated list such as the thread counts must be put in quotes there).

9.1 Build

```
cd Lab1
dotnet build -c Release
function lab1 { dotnet bin/Release/net10.0/Lab1.dll @args }   # shortcut used below
lab1 help
```

9.2 Generating the Data

```
mkdir results/logs
lab1 info > results/logs/info.txt
lab1 gen     # data/numbers.txt: 50 000 000 integers (seed 2026) + numbers.txt.counts
lab1 split   # data/parts/part_1.txt .. part_8.txt (6 250 000 lines each) +
expected.counts
```

9.3 Reproducing the Measurements

```
lab1 bench --task 1a --runs 5 --csv results/task1a.csv
lab1 bench --task 1b --threads 16 --runs 1 --warmup 0 --csv results/task1b.csv
lab1 bench --task 1b --threads "1,2,4,8,16,32,64" --runs 3 --warmup 0 --repeat 1 --csv
results/task1b-sweep.csv
lab1 bench --task 1c --runs 3 --csv results/task1c.csv
lab1 bench --task 1c --threads "1,4,16" --runs 1 --batch 1 --csv results/task1c-batch1.csv
lab1 bench --task 1c --threads "1,4,16" --runs 3 --capacity 0 --csv results/task1c-
unbounded.csv
lab1 bench --task 1c --threads 1 --runs 1 --warmup 0 --batch 1 --capacity 0 --csv
results/task1c-batch1-unbounded.csv
lab1 bench --task 2 --threads "1,2,4" --dir data/parts --csv results/task2-ssd-warm.csv
lab1 bench --task 2 --threads "1,2,4" --runs 1 --pause --dir E:\lab1 --csv
results/task2.csv
```

`bench` starts a new child process for every run, so the peak memory belongs to that one configuration. Task 1a and 1c start with one unrecorded warm-up run, so that the file is in the OS page cache and every recorded run reads it from memory and not from the SSD. For the USB run, the drive was ejected, unplugged and plugged in again before every run, and the 8 files and `expected.counts` had been copied to `E:\lab1` beforehand.